

UNIVERSIDAD PANAMERICANA

Facultad De Ingeniería y Ciencias Aplicadas

Ingeniería en Sistemas y Tecnologías de la Información y la Comunicación

Programación V/ Base de datos 2



Proyecto HIPSTAGRAM – Fase 3

(Seguridad e Integridad Transaccional)

Integrantes:

Douglas Esaú Catú Otzoy (000140060)

Kevin Eliazar Herrera Alburez (000113637)

Priscila Andrea Güitz del Cid (000139626)

Carlos Enrique Otoniel Bautista Sarazua (000133047)

Herbert David Estrada Trujillo (000141126)

Chimaltenango, Chimaltenango, junio 2026

SEGURIDAD E INTEGRIDAD TRANSACCIONAL

Proyecto HIPSTAGRAM

Plataforma de Red Social con Microservicios

Versión	2.0
Fecha	Junio 2026
Equipo	Equipo de Desarrollo — 5 integrantes
Curso	Proyecto General — Ing. Mario Obed Morales Guitz

Índice

Introducción	1
1. Roles y usuarios de base de datos	2
1.1 Diseño de roles	2
1.2 Transacciones: BEGIN/ COMMIT/ ROLLBAK/SAVEPOINT	8
2. Que es una transacción de base de datos	13
2.1. Porque Hipstagram necesita transacciones	13
2.2. Flujo T1 publicar foto.....	14
2.3 Flujo T2 registrar voto.....	15
2.4 Flujo T3 moderación de publicación.....	16

Introducción

En esta sección se detalla el diseño y la implementación de los mecanismos de seguridad, integridad y control transaccional que rigen la capa de persistencia del ecosistema Hipstagram. Dado que la plataforma opera bajo una arquitectura de microservicios, la robustez de la base de datos es fundamental para garantizar la resiliencia del sistema y la coherencia de la información ante fallos operativos o intentos de acceso no autorizado.

La especificación técnica aquí presentada se articula sobre tres ejes fundamentales de la ingeniería de datos:

1. **Seguridad Perimetral e Interna (PoLP):** Se describe la creación de roles de base de datos fundamentados en el **Principio de Mínimo Privilegio**. Mediante el uso estratégico de comandos GRANT y REVOKE, se asegura que el usuario de aplicación (app_role) carezca de facultades de lectura sobre la bitácora de auditoría, mitigando el riesgo de exposición de datos sensibles en caso de una vulnerabilidad en el backend.
2. **Integridad Referencial y Manejo de Excepciones:** Se documenta la configuración de llaves foráneas (Foreign Keys) bajo la política ON DELETE RESTRICT en tablas de alta criticidad, asegurando que el motor de base de datos bloquee eliminaciones que comprometan la consistencia sistémica. Este control se complementa con la interceptación del error **SQLSTATE 23503** en la capa de servicios, permitiendo transformar excepciones técnicas en respuestas controladas hacia el usuario final.
3. **Control Transaccional Avanzado (ACID):** Se analiza la orquestación de flujos transaccionales complejos mediante el uso de sentencias BEGIN, COMMIT y ROLLBACK. Se otorga especial énfasis a la implementación de **SAVEPOINT** como mecanismo de recuperación parcial, permitiendo que el sistema preserve la atomicidad en operaciones de negocio críticas mientras gestiona de forma aislada fallos en procesos secundarios o no bloqueantes.

A través de esta implementación, Hipstagram no solo asegura el cumplimiento de las reglas de negocio, sino que establece un entorno de datos altamente auditable, seguro y resistente a la concurrencia masiva.

1. Roles y usuarios de base de datos

1.1 Diseño de roles

El aprovisionamiento de accesos en el motor de base de datos se fundamenta en el Principio de Menor Privilegio (PoLP). Esta estrategia garantiza que ninguna identidad de servicio o perfil administrativo disponga de facultades que excedan las responsabilidades operativas estrictamente necesarias para su función. Para materializar este modelo, se han configurado tres roles jerárquicos con perímetros de acción delimitados:

- **Rol de Administración de Infraestructura (dba_role):** Vinculado al usuario `hip_dba_user`, este rol ostenta autoridad total (ALL PRIVILEGES) sobre la totalidad de las tablas y secuencias del sistema. Su operatividad está reservada exclusivamente para la ejecución de migraciones de datos y tareas de mantenimiento evolutivo del esquema relacional.
- **Rol de Operación de Aplicación (app_role):** Asignado al usuario `hip_app_user` para la interacción con el *backend* (Node.js). Está facultado para ejecutar transacciones de negocio (SELECT, INSERT, UPDATE, DELETE) sobre las tablas de dominio. Como control de seguridad crítico, se ha implementado un **REVOKE explícito de privilegios de lectura (SELECT)** sobre la tabla de auditoría; de este modo, el servicio solo puede aprovisionar nuevos registros (INSERT), blindando el historial del sistema contra lecturas no autorizadas.
- **Rol de Auditoría Administrativa (admin_read_role):** Vinculado al usuario `hip_admin_user`, este perfil opera bajo un paradigma de solo lectura. Su alcance funcional se limita a la ejecución de consultas (SELECT) sobre la bitácora de auditoría y el registro de usuarios, careciendo de acceso a las tablas operativas de negocio para asegurar la integridad en la generación de reportes y supervisión técnica.

Este esquema de segregación de privilegios asegura que la superficie de ataque se mantenga mínima, protegiendo la persistencia de datos frente a potenciales compromisos en la capa de aplicación.

Rol	Usuarios BD	Propósito y restricciones
Dbá role	Hip dba user	Acceso total: ALL PRIVILEGES en todas las tablas y secuencias. Solo para migraciones y mantenimiento de esquema.
App role	Hip app user	Backend Node.js: SELECT/INSERT/UPDATE/DELETE en tablas de negocio. Solo INSERT en audit_logs. REVOKE explícito de SELECT en audit_logs.
Admin read role	Hip admin user	Solo lectura: SELECT en audit_logs y usuarios. Sin acceso a tablas de negocio. Para reportes y auditoría

Script: crear roles y usuarios

```

16
17     -- Crear los roles
18 CREATE ROLE dba_role;
19 CREATE ROLE app_role;
20 CREATE ROLE admin_read_role;
21
22 -- Crear usuarios de BD y asignarles su rol
23 CREATE USER hip_app_user WITH PASSWORD 'App_Secure_2026!';
24 CREATE USER hip_admin_user WITH PASSWORD 'Admin_Read_2026!';
25 CREATE USER hip_dba_user WITH PASSWORD 'Dbá_Master_2026!';
26
27 GRANT app_role TO hip_app_user;
28 GRANT admin_read_role TO hip_admin_user;
29 GRANT dba_role TO hip_dba_user;

```

Grant / revoke por rol

Revocar privilegios del public

Antes de asignar privilegios, se revoca el acceso por defecto que PostgreSQL otorga al rol PUBLIC:

```

31
32 REVOKE CREATE ON SCHEMA public FROM PUBLIC;
33 REVOKE ALL ON ALL TABLES IN SCHEMA public FROM PUBLIC;
34

```

Db_role – acceso total

```
36 GRANT CONNECT ON DATABASE Hipstagram TO dba_role;
37 GRANT USAGE ON SCHEMA public TO dba_role;
38 GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO dba_role;
39 GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO dba_role;
40 GRANT CREATE ON SCHEMA public TO dba_role;
```

App_role – backend Node.js (sin select en auditoria)

El backend puede operar con todas las tablas de negocio, pero SOLO puede insertar en audit_logs. No puede leer el historial de auditoría. Esto garantiza que una brecha en el backend no exponga el log completo del sistema:

```
42
43 GRANT CONNECT ON DATABASE Hipstagram TO app_role;
44 GRANT USAGE ON SCHEMA public TO app_role;
45
46 -- Operaciones de negocio completas
47 GRANT SELECT, INSERT, UPDATE, DELETE
48 ON usuarios, publicacion, hashtags,
49 publicacion_hashtags, votos, comentarios
50 TO app_role;
51
52 -- Solo puede INSERTAR en audit_logs (nunca leer)
53 GRANT INSERT ON auditoria TO app_role;
54
55 --| REVOKE EXPLÍCITO: app_role NO puede SELECT en audit_logs
56 REVOKE SELECT ON auditoria FROM app_role;
57
58 -- Secuencias para SERIAL / autoincrement
59 GRANT USAGE, SELECT ON SEQUENCE
60 usuarios_id_seq, publicacion_id_seq,
61 hashtags_id_seq, votos_id_seq,
62 comentarios_id_seq, auditoria_id_seq
63 TO app_role;
64
```

Admin read role – solo lectura en auditoria

Este rol es exclusivo para consultas administrativas y de auditoría. Solo puede leer, nunca modificar:

```
66
67 GRANT CONNECT ON DATABASE Hipstagram TO admin_read_role;
68 GRANT USAGE ON SCHEMA public TO admin_read_role;
69
70 -- Solo SELECT en audit_logs (su único propósito)
71 GRANT SELECT ON auditoria TO admin_read_role;
72 GRANT SELECT ON usuarios TO admin_read_role;
73
74 -- Sin acceso al resto de tablas de negocio
75 REVOKE ALL ON publicacion, hashtags, publicacion_hashtags,
76 votos, comentarios
77 FROM admin_read_role;
```

Verificación de privilegios

Consulta para confirmar que la configuración es correcta:

```
79
80 -- Ver todos los privilegios por rol
81 SELECT grantee, table_name,
82        string_agg(privilege_type, ', ' ) AS privilegios
83 FROM   information_schema.role_table_grants
84 WHERE  table_schema = 'public'
85        AND grantee IN ('app_role','admin_read_role','dba_role')
86 GROUP BY grantee, table_name
87 ORDER BY grantee, table_name;
88
```

Resultado esperado (extracto):

Grantee	Tabla	Privilegios
app_role	publicacion	DELETE, INSERT, SELECT, UPDATE
app_role	audit_logs	INSERT (solo insertar — sin SELECT )
admin_read_role	audit_logs	SELECT (solo leer )
admin_read_role	publicacion	(sin privilegios )
dba_role	(todas)	ALL PRIVILEGES

```
89
90 -- Verificar que app_role NO tiene SELECT en auditoria
91 -- Resultado esperado: 0 filas
92 SELECT grantee, table_name, privilege_type
93 FROM   information_schema.role_table_grants
94 WHERE  table_name = 'auditoria'
95        AND grantee = 'app_role'
96        AND privilege_type = 'SELECT';
```


Foreign keys con ON DELETE RESTRICT

Tabla de FK criticas

Constraint	Relación hijo -> padre	On delete	Efecto
fk_pub_usuario	publicacion → usuarios	RESTRICT	No se puede borrar un usuario que tenga publicaciones
fk_com_pub	comentarios → publicacion	RESTRICT	No se puede borrar una publicación que tenga comentarios
fk_voto_pub	votos → publicacion	RESTRICT	No se puede borrar una publicación que tenga votos
fk_ph_pub	publicacion_hashtags → publicacion	CASCADE	Al borrar publicación, se borran sus hashtags automáticamente
fk_audit_usr	auditoria → usuarios	CASCADE	Al borrar usuario, el log se conserva con usuario_id = NULL

DDL DE LAS FK CRITICAS

```
99  -- usuarios → publicacion (RESTRICT)
100  CONSTRAINT fk_pub_usuario
101  FOREIGN KEY (usuario_id)
102  REFERENCES usuarios(id)
103  ON DELETE RESTRICT
104  ON UPDATE CASCADE,
105
106  -- publicacion → comentarios (RESTRICT)
107  CONSTRAINT fk_com_pub
108  FOREIGN KEY (publicacion_id)
109  REFERENCES publicacion(id)
110  ON DELETE RESTRICT,
111
112  -- publicacion → votos (RESTRICT)
113  CONSTRAINT fk_voto_pub
114  FOREIGN KEY (publicacion_id)
115  REFERENCES publicacion(id)
116  ON DELETE RESTRICT,
```

Demostración error 23503 al intentar DELETE

Al intentar eliminar una publicación que tiene comentarios o votos, PostgreSQL lanza el error SQLSTATE 23503:

```

119 -- Intento de borrar publicación con comentarios existentes:
120 DELETE FROM publicacion WHERE id = 1;
121
122 -- PostgreSQL responde:
123 ERROR: update or delete on table "publicacion" violates
124       foreign key constraint "fk_com_pub" on table "comentarios"
125 DETAIL: Key (id)=(1) is still referenced from table "comentarios".
126 SQLSTATE: 23503
127 |

```

BACKEND Node.js captura del error 23503 -> HTTP 409

El backend captura el código de error específico de PostgreSQL y devuelve una respuesta controlada en lugar de exponer el error técnico al cliente:

```

129 -- routes/posts.js - DELETE /posts/:id
130 router.delete('/:id', requireAuth, async (req, res) => {
131   try {
132     // Verificar que el post existe y es del usuario (o es ADMIN)
133     const pRes = await query(
134       'SELECT * FROM publicacion WHERE id=$1', [postId]
135     );
136     if (!pRes.rows[0])
137       return res.status(404).json({ error: 'Publicación no encontrada.' });
138
139     if (pRes.rows[0].usuario_id !== userId && req.user.rol !== 'ADMIN')
140       return res.status(403).json({
141         error: 'No puedes eliminar una publicación que no es tuya.'
142       });
143
144     await query('DELETE FROM publicacion WHERE id=$1', [postId]);
145
146   } catch (err) {
147     --Capturar violación de FK (SQLSTATE 23503)
148     if (err.code === '23503') {
149       return res.status(409).json({
150         error: 'No se puede eliminar porque tiene comentarios o votos.',
151         code: 'FK_VIOLATION'
152       });
153     }
154     return res.status(500).json({ error: 'Error interno del servidor.' });
155   }
156 });

```

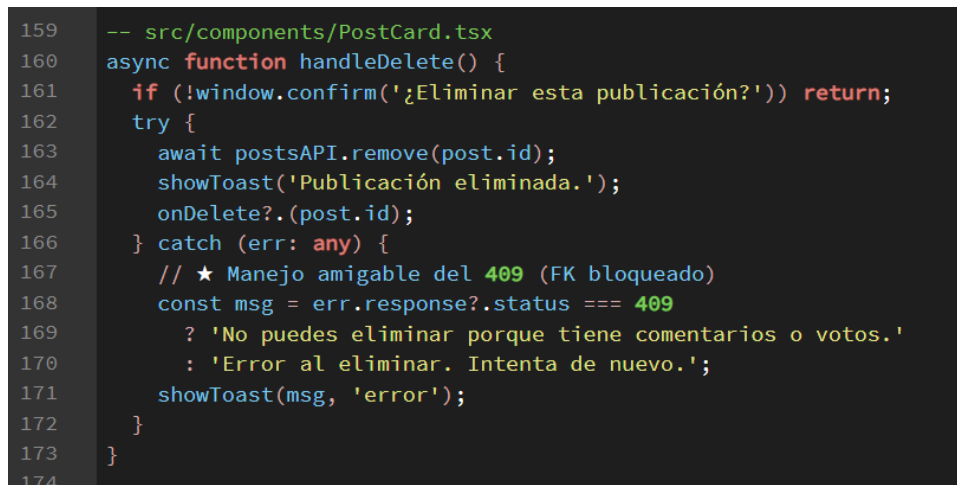
Tabla de respuestas HTTP del endpoint DELETE /posts/:id:

HTTP	Causa	Body JSON
200	Eliminado correctamente	{ "message": "Publicación eliminada." }
403	No es el autor	{ "error": "No puedes eliminar una publicación que no es tuya." }
404	Post no encontrado	{ "error": "Publicación no encontrada." }
409	FK violation (23503)	{ "error": "No se puede eliminar porque tiene comentarios o votos.", "code": "FK_VIOLATION" }

Frontend react: mensaje amigable al usuario

El componente PostCard.tsx en el frontend captura el HTTP 409 y muestra un mensaje comprensible al usuario, sin exponer detalles técnicos:

```
159 -- src/components/PostCard.tsx
160 async function handleDelete() {
161   if (!window.confirm('¿Eliminar esta publicación?')) return;
162   try {
163     await postsAPI.remove(post.id);
164     showToast('Publicación eliminada.');
```



```
165     onDelete?.(post.id);
166   } catch (err: any) {
167     // ★ Manejo amigable del 409 (FK bloqueado)
168     const msg = err.response?.status === 409
169       ? 'No puedes eliminar porque tiene comentarios o votos.'
170       : 'Error al eliminar. Intenta de nuevo.';
171     showToast(msg, 'error');
172   }
173 }
174
```

El usuario ve el mensaje en un toast de color rojo en la esquina inferior derecha. Nunca ve el código SQL ni el SQLSTATE 23503.

1.2 Transacciones: BEGIN/ COMMIT/ ROLLBAK/SAVEPOINT

Porque transacciones explícitas

- PostgreSQL en modo autocommit confirmaría cada INSERT por separado. Si el segundo falla, el primero ya quedó guardado dejando datos inconsistentes.
- Con transacciones explícitas, todos los cambios relacionados se guardan juntos o no se guarda ninguno (atomicidad).
- SAVEPOINT permite hacer rollback parcial: si los hashtags fallan, la publicación se guarda de todas formas sin abortar todo.
- El pool de conexiones de pg requiere obtener un client dedicado con pool.connect() para que todas las queries estén en la misma transacción.

Flujo T1 publicar foto

Tablas involucradas: publicacion → publicacion_hashtags → hashtags → auditorio

Paso	Acción SQL	Si falla...
1. BEGIN	Inicia transacción	—
2. INSERT publicacion	Guarda la foto	ROLLBACK completo → HTTP 500
3. SAVEPOINT sp_hashtags	Punto de recuperación	—
4. INSERT hashtags (ON CONFLICT DO NOTHING)	Crea hashtag si no existe	ROLLBACK TO sp_hashtags (foto se guarda sin hashtags)
5. INSERT publicacion_hashtags	Vincula hashtag	ROLLBACK TO sp_hashtags
6. SAVEPOINT sp_audit	Punto antes de log	—
7. INSERT audit_logs	Registra acción	ROLLBACK TO sp_audit (no cancela la foto)
8. COMMIT	Todo guardado	Atómico

Código Node.js del flujo T1

```
const client = await pool.connect();
try {
  await client.query('BEGIN');

  -- Paso 2: Insertar publicación
  const pRes = await client.query(
    `INSERT INTO publicacion (usuario_id, imagen_url, descripcion, estado)
    VALUES ($1, $2, $3, 'PUBLICADO') RETURNING *`,
    [userId, imagenUrl, descripcion]
  );
  const post = pRes.rows[0];

  -- Paso 3: SAVEPOINT antes de hashtags
  await client.query('SAVEPOINT sp_hashtags');
  try {
    for (const nombre of hashtags) {
      await client.query(
        'INSERT INTO hashtags (nombre) VALUES ($1) ON CONFLICT (nombre) DO NOTHING',
        [nombre]
      );
    }
    const hRes = await client.query(
      'SELECT id FROM hashtags WHERE nombre=$1', [nombre]
    );
    await client.query(
      'INSERT INTO publicacion_hashtags VALUES ($1,$2) ON CONFLICT DO NOTHING',
      [post.id, hRes.rows[0].id]
    );
  }
}
```

```

    } catch {
      -- Hashtags fallaron: revertir solo esa parte
      await client.query('ROLLBACK TO SAVEPOINT sp_hashtags');
    }

    -- Paso 6: SAVEPOINT antes de auditoría
    await client.query('SAVEPOINT sp_audit');
    try {
      await client.query(
        `INSERT INTO audit_logs (usuario_id, accion, tabla_afectada, detalles, direccion_
        VALUES ($1, 'POST_CREADO', 'publicacion', $2, $3)`,
        [userId, JSON.stringify({post_id: post.id}), ip]
      );
    } catch {
      -- Auditoría falló: no crítico, no cancela la publicación
      await client.query('ROLLBACK TO SAVEPOINT sp_audit');
    }

    await client.query('COMMIT'); // Todo guardado
    return res.status(201).json({ post });
  } catch (err) {
    await client.query('ROLLBACK'); // Error grave: revertir todo
    return res.status(500).json({ error: 'Error al crear la publicación.' });
  } finally {
    client.release(); // Devolver conexión al pool
  }
}

```

Flujo T2: votar (upsert + actualización de contadores)

Tablas: votos (upsert) + publicacion (UPDATE contadores) + audit_logs

El voto usa INSERT ... ON CONFLICT DO UPDATE para manejar en una sola query tanto el voto nuevo como el cambio de reacción. FOR UPDATE bloquea la fila para evitar condición de carrera en concurrencia:

```

const client = await pool.connect();
try {
  await client.query('BEGIN');

  -- Verificar que no sea su propio post (regla de negocio)
  const pRes = await client.query(
    `SELECT usuario_id FROM publicacion WHERE id=$1`, [postId]
  );
  if (pRes.rows[0].usuario_id === userId) {
    await client.query('ROLLBACK');
    return res.status(400).json({
      error: 'No puedes votar tu propia publicación.'
    });
  }

  -- UPSERT: inserta el voto o cambia el tipo si ya existía
  await client.query(
    `INSERT INTO votos (publicacion_id, usuario_id, tipo_voto)
    VALUES ($1, $2, $3)
    ON CONFLICT (usuario_id, publicacion_id)
    DO UPDATE SET tipo_voto = EXCLUDED.tipo_voto`,
    [postId, userId, tipoVoto]
  );
}

```

```

-- Recalcular contadores exactos desde la tabla votos
await client.query(
  `UPDATE publicacion SET
    likes_count      = (SELECT COUNT(*) FROM votos
                        WHERE publicacion_id=$1 AND tipo_voto=1),
    dislikes_count = (SELECT COUNT(*) FROM votos
                        WHERE publicacion_id=$1 AND tipo_voto=0)
  WHERE id=$1`,
  [postId]
);

```

```

-- Auditoría del voto
await client.query(
  `INSERT INTO audit_logs (usuario_id, accion, tabla_afectada, detalles)
  VALUES ($1, 'VOTO_REGISTRADO', 'votos', $2)`,
  [userId, JSON.stringify({post_id: postId, tipo: tipoVoto})]
);

await client.query('COMMIT');

} catch (err) {
  await client.query('ROLLBACK');
  -- El frontend revierte el optimistic UI automáticamente
  return res.status(500).json({ error: 'Error al registrar el voto.' });
} finally {
  client.release();
}

```

Fujo T3 moderación (estado del post + auditoria obligatoria)

```

const client = await pool.connect();
try {
  await client.query('BEGIN');

  await client.query('SAVEPOINT sp_moderacion');

  -- Paso 1: Cambiar el estado del post
  const result = await client.query(
    `UPDATE publicacion SET estado=$1 WHERE id=$2 RETURNING estado`,
    [nuevoEstado, postId]
  );

  -- Si el post no existe, hacer rollback
  if (result.rowCount === 0) {
    await client.query('ROLLBACK TO SAVEPOINT sp_moderacion');
    await client.query('ROLLBACK');
    return res.status(404).json({ error: 'Publicación no encontrada.' });
  }

  -- Paso 2: Registrar en auditoría (obligatorio)
  await client.query(
    `INSERT INTO audit_logs
    (usuario_id, accion, tabla_afectada, detalles, direccion_ip)
    VALUES ($1, $2, 'publicacion', $3, $4)`,
    [

```

```

-- Paso 2: Registrar en auditoría (obligatorio)
await client.query(
  `INSERT INTO audit_logs
    (usuario_id, accion, tabla_afectada, detalles, direccion_ip)
  VALUES ($1, $2, 'publicacion', $3, $4)\`,
  [
    adminId,
    action === 'approve' ? 'ADMIN_APROBAR_POST' : 'ADMIN_BLOQUEAR_POST',
    JSON.stringify({
      post_id:      postId,
      estado_nuevo: nuevoEstado,
      moderado_por: adminUsername
    }),
    ip
  ]
);

await client.query('COMMIT'); -- Estado + auditoría juntos
return res.json({ message: `Post ${nuevoEstado.toLowerCase()}.` });
} catch (err) {
  -- Si audit_logs falla, revertir también el cambio de estado
  await client.query('ROLLBACK');
  return res.status(500).json({ error: 'Error en moderación.' });
} finally {
  client.release();
}

```

Demo en psql – SAVEPOINT con rollback parcial

Esta demostración muestra el comportamiento de SAVEPOINT directamente en la consola de PostgreSQL:

```

-- Inicio de transacción
BEGIN;

-- La publicación se inserta correctamente
INSERT INTO publicacion (usuario_id, imagen_url, descripcion)
VALUES (2, '/uploads/demo.jpg', 'Demo SAVEPOINT');
-- (1 row)

SAVEPOINT sp_hashtags;

-- Intentar insertar hashtag inválido (nombre vacío, viola CHECK)
INSERT INTO hashtags (nombre) VALUES ('');
-- ERROR: valor nulo en columna / violación de restricción

-- Revertir SOLO los hashtags, conservar la publicación
ROLLBACK TO SAVEPOINT sp_hashtags;

-- La publicación SIGUE en la transacción
SELECT descripcion FROM publicacion WHERE imagen_url='/uploads/demo.jpg';
-- 'Demo SAVEPOINT' ← sigue visible dentro de la transacción

COMMIT;
--| Publicación guardada, hashtag inválido descartado

```

2. Que es una transacción de base de datos

Una transacción es un conjunto de operaciones de base de datos que se tratan como una única unidad de trabajo. La idea fundamental es sencilla pero poderosa: o todas las operaciones del conjunto se completan con éxito y los cambios se guardan de forma permanente, o ninguna de ellas se aplica y la base de datos queda exactamente como estaba antes. No existe un estado intermedio.

Para entender por qué esto es necesario, consideremos un ejemplo cotidiano fuera del mundo de la tecnología: una transferencia bancaria. Cuando una persona transfiere dinero de su cuenta a otra, ocurren dos operaciones: primero se resta el dinero de la cuenta origen, y después se suma el mismo monto a la cuenta destino. Si el sistema falla entre estas dos operaciones, el dinero habría desaparecido de la primera cuenta sin llegar a la segunda. Una transacción garantiza que ambas operaciones ocurran juntas o ninguna ocurra.

En Hipstagram ocurre exactamente lo mismo: publicar una foto implica guardar el registro de la publicación Y registrar sus hashtags Y anotar la acción en el log de auditoría. Si cualquiera de estas tres operaciones falla a la mitad, la base de datos quedaría en un estado incoherente. Las transacciones evitan este problema.

2.1. Porque Hipstagram necesita transacciones

Sin transacciones, los flujos de Hipstagram quedarían expuestos a varios problemas de integridad. A continuación, se presentan los escenarios problemáticos reales y cómo las transacciones los resuelven:

Flujo	Problema sin transacción	Solución con transacción
Publicar foto	La foto se guarda, pero falla el INSERT de hashtags. La publicación queda en BD sin sus etiquetas. El log de auditoría no se crea. No hay forma de saber qué pasó.	BEGIN + SAVEPOINT garantiza que, si los hashtags fallan, el error se captura sin perder la foto. El COMMIT final solo ocurre si todo está en orden.
Votar	El voto se inserta en la tabla votos, pero el UPDATE de likes_count falla. El contador en pantalla nunca cambia, aunque el voto sí quedó guardado. Inconsistencia visible al usuario.	Transacción asegura que el INSERT del voto y el UPDATE del contador siempre ocurren juntos. Si uno falla, el otro se revierte.
Moderación	El estado del post cambia a BLOQUEADO pero el INSERT en auditoria falla. El administrador tomó una acción sin evidencia. Problema legal y de trazabilidad.	La transacción une el cambio de estado y el registro de auditoría. Si el auditoria falla, el estado del post vuelve a su valor original automáticamente.

2.2. Flujo T1 publicar foto

Este flujo representa la operación más compleja del sistema al involucrar cuatro tablas con distintos niveles de criticidad. El diseño orquesta la persistencia de la imagen (crítica), los metadatos de indexación (opcionales) y el registro de auditoría (transversal). La implementación de puntos de control (*Savepoints*) asegura que la fotografía se conserve incluso si los procesos de etiquetado o logueo presentan fallos inesperados.

Tablas involucradas y responsabilidad de cada una

Tabla	Operación	Criticidad	Descripción
publicacion	INSERT	CRÍTICA	Registra la foto con su URL, descripción y estado. Si este INSERT falla, toda la transacción debe cancelarse porque sin la publicación no tiene sentido el resto.
hashtags	INSERT (ON CONFLICT DO NOTHING)	OPCIONAL	Crea el hashtag si no existe todavía. Si ya existe, lo ignora sin error. Si falla por algún motivo inesperado, se usa ROLLBACK TO SAVEPOINT para ignorar el error sin perder la foto.
publicacion_hashtags	INSERT	OPCIONAL	Vincula la publicación con el hashtag. Tabla puente de la relación N:M. Igual que hashtags: si falla, solo se descarta ese hashtag.
auditoria	INSERT	NO CRÍTICA	Registra la acción en el log de auditoría. Importante para trazabilidad, pero si falla (por ejemplo, por un problema temporal de permisos), la publicación NO debe perderse. Por eso tiene su propio SAVEPOINT.

Diagrama de decisión del flujo T1

#	Acción SQL / Node.js	Punto control	de Si falla → acción a tomar
1	BEGIN	—	Inicia la transacción. Ningún cambio es visible aún.
2	INSERT publicacion	—	Éxito → continuar Error → ROLLBACK total, HTTP 500
3	SAVEPOINT sp_hashtags	sp_hashtags	Marca el punto de recuperación para hashtags
4	INSERT hashtags (ON CONFLICT...)	—	Éxito → continuar Error → ROLLBACK TO sp_hashtags
5	INSERT publicacion hashtags	—	Éxito → continuar Error → ROLLBACK TO sp_hashtags

6	SAVEPOINT sp_audit	sp_audit	Marca el punto de recuperación para auditoría
7	INSERT audit_logs	—	Éxito → continuar Error → ROLLBACK TO sp_audit (foto se conserva)
8	COMMIT	—	Todo guardado permanentemente. HTTP 201 al cliente.

2.3 Flujo T2 registrar voto

Para mitigar condiciones de carrera en entornos de alta concurrencia —donde múltiples usuarios interactúan simultáneamente con el mismo recurso— se emplea una estrategia de UPSERT (ON CONFLICT DO UPDATE). Esta técnica asegura la idempotencia de la operación; adicionalmente, en lugar de incrementar contadores de forma manual, el sistema recalcula los valores exactos directamente desde la tabla de votos, garantizando que el recuento visual refleje siempre el estado real de la base de datos

Que es una condición de carrera

Una condición de carrera ocurre cuando dos operaciones simultáneas leen el mismo valor, cada una le suma 1, y ambas escriben su resultado. El valor final es el mismo que si solo una hubiera votado, cuando en realidad votaron dos. En una red social esto significaría contadores incorrectos.

Ejemplo del problema: la publicación tiene 10 likes. Usuario A y Usuario B votan al mismo tiempo. Ambos leen 10. Ambos calculan $10+1=11$. Ambos escriben 11. El resultado es 11 en lugar del correcto que sería 12.

La solución de Hipstagram usa dos mecanismos combinados: el UPSERT con ON CONFLICT DO UPDATE para manejar el voto en una sola operación atómica, y el recálculo de contadores directamente desde la tabla de votos en lugar de incrementar el valor existente:

Casos que maneja la transacción

Caso	Situación	Operación SQL	Resultado esperado
Voto nuevo	El usuario nunca ha votado esta publicación	INSERT INTO votos (ON CONFLICT no activa)	Se inserta la fila. Contador sube en 1.
Cambio de voto	El usuario ya votó y quiere cambiar (like → dislike)	INSERT INTO votos ON CONFLICT DO UPDATE SET tipo_voto = EXCLUDED.tipo_voto	Se actualiza el tipo votó existente. Un contador sube y la otra baja.

Voto idempotente	El usuario hace clic en el mismo voto que ya tenía	INSERT INTO votos ON CONFLICT DO UPDATE (mismo valor)	Sin cambio neto en los contadores. El recálculo lo maneja.
Voto propio	El usuario intenta votar su propia publicación	Validación en Node.js antes del BEGIN	HTTP 400. La transacción nunca se abre.

Diagrama de decisión del flujo T2

#	Acción SQL / Node.js	Punto de control	de Si falla → acción a tomar
1	Validar: ¿es el autor?	—	Si sí → HTTP 400, no abrir transacción. Si no → continuar.
2	BEGIN	—	Inicia la transacción de voto.
3	INSERT votos ON CONFLICT DO UPDATE	—	Éxito → voto insertado o actualizado Error → ROLLBACK
4	UPDATE publicacion (recalcular contadores)	—	Contadores actualizados Error → ROLLBACK (voto también se revierte)
5	INSERT audit_logs	—	Acción registrada Error → ROLLBACK todo
6	COMMIT	—	Voto y contadores guardados atómicamente.

2.4 Flujo T3 moderación de publicación

La moderación es el flujo donde la atomicidad es más importante desde el punto de vista legal y de trazabilidad. Un administrador que cambia el estado de una publicación sin dejar registro en audit_logs ha tomado una acción sin evidencia. Para el sistema de Hipstagram, esto es inaceptable: el estado del post y su registro de auditoría son inseparables.

Porque son inseparables el estado y la auditoria

Imagina que el administrador bloquea una publicación inapropiada. La BD cambia el estado a BLOQUEADO, pero justo en ese momento el servidor tiene un error y el INSERT en audit_logs no se ejecuta. Resultado: la publicación está bloqueada pero no hay registro de quién la bloqueó, cuándo, ni por qué. No hay evidencia.

La solución es incluir ambas operaciones en la misma transacción con un SAVEPOINT que las une. Si el audit_logs falla, el cambio de estado también se revierte. Esto obliga al sistema a garantizar que ambas cosas ocurren siempre juntas.

Estados posibles de una publicación

Estado	Visible feed	en	Quién puede cambiar	Registro en audit_logs
PUBLICADO	Sí		Usuario (al crear), Admin	POST_CREADO al publicar; ADMIN_APROBAR al desbloquear
BLOQUEADO	No		Solo ADMIN	ADMIN_BLOQUEAR_POST con detalle del motivo
PENDIENTE	No		Sistema (si se activa moderación previa)	ADMIN_APROBAR o ADMIN RECHAZAR POST

Diagrama de decisión del flujo T3

#	Acción SQL / Node.js	Punto de control	Si falla → acción a tomar
1	Validar rol ADMIN	—	Si no es ADMIN → HTTP 403. No abrir transacción.
2	BEGIN	—	Inicia la transacción de moderación.
3	SAVEPOINT sp_moderacion	sp_moderacion	Punto de control antes de modificar datos.
4	UPDATE publicacion SET estado = \$1	—	1 fila afectada → continuar 0 filas (no existe) → ROLLBACK TO sp_moderacion + ROLLBACK
5	INSERT audit_logs (acción + detalles)	—	Log creado → continuar Error → ROLLBACK TO sp_moderacion (estado del post revierte también)
6	COMMIT	—	Estado y auditoría guardados juntos. HTTP 200.

Patron de código Pool de conexiones y transacciones

Un detalle técnico importante en Node.js con la librería pg es que las transacciones REQUIEREN usar una conexión dedicada obtenida con `pool.connect()`. Si se usa la función `query()` del pool directamente, cada llamada puede usar una conexión diferente y la transacción no funciona.

Método	¿Cuándo usar?	¿Soporta transacciones?	Ejemplo
<code>query(sql, params)</code>	Queries simples de lectura o escritura individual	No. Cada llamada puede ir en conexión diferente	<code>SELECT</code> de posts, <code>INSERT</code> de comentario simple
<code>pool.connect() + client.query()</code>	Flujos que requieren múltiples operaciones atómicas	Sí. Toda la transacción usa la misma conexión	Publicar foto, votar, moderar